C++ OBJECT-ORIENTED PROGRAMMING

GameQuest

Gaming Tournament and Player Ranking System

A mini project featuring a **Valorant** Champions-style tournament

Concepts demonstrated	Classes and Objects, Inheritance, Polymorphism, Operator Overloading, File Handling, Exception Handling
Language	C++17 (compiled with g++)
Theme / Game	Valorant (agents, maps, pro and casual players)
Student name	Sakshi
UID	25bca10148
Section	25BCA3B
Submission date	October 2026

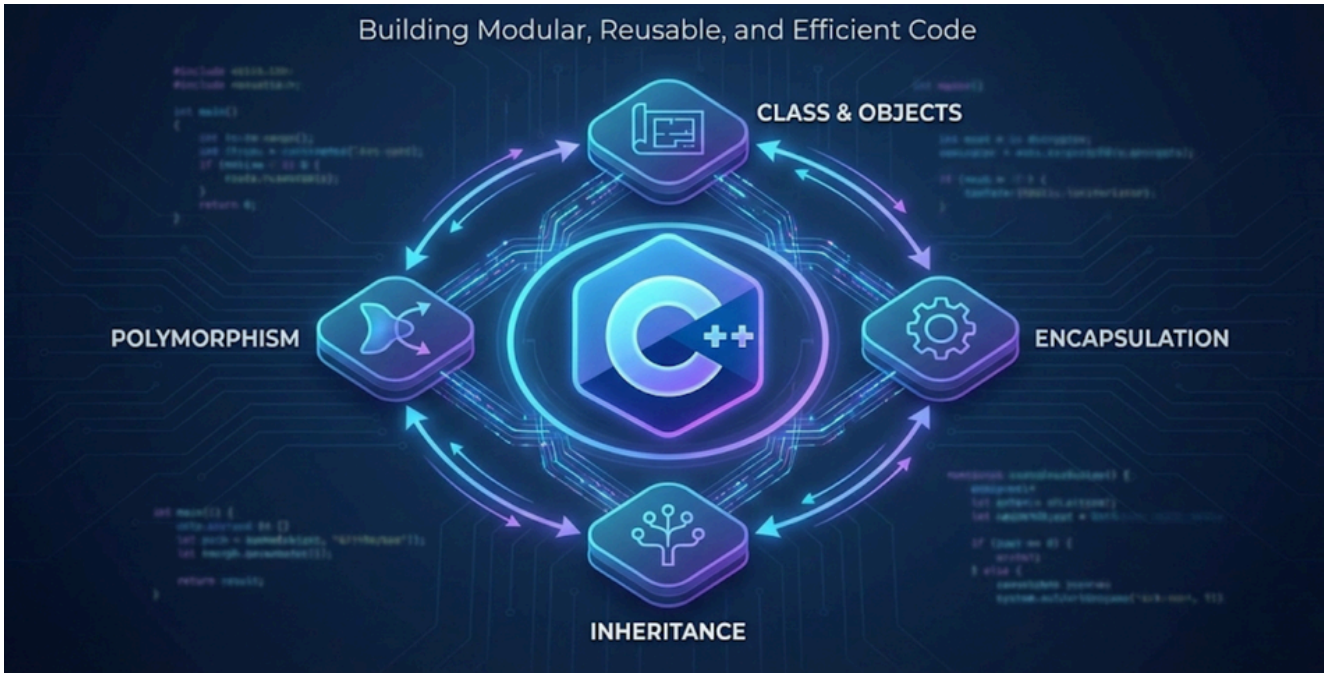


Table of Contents

1. Introduction 3

2. System Design 4

3. Scoring and Reward Rules 5

4. Implementation 6

5. Concepts in Detail 11

6. Program Output 12

7. Saved Files 13

8. Testing 14

9. Conclusion and Future Scope 15

Abstract

GameQuestis a console-based tournament management system written in C++. It registers players, schedules matches on Valorant maps, validates and records match statistics (kills, deaths and assists), ranks players on a live leaderboard, calculates rewards and saves the final results to text and CSV files. The project is deliberately built around six core object-oriented programming ideas: classes, inheritance, polymorphism, operator overloading, file handling and exception handling. Two player types, **CasualPlayer** and **ProPlayer**, share one virtual interface but score and get rewarded differently, which makes the polymorphism visible in the output. Invalid input such as negative or non-numeric scores is rejected by custom exceptions before it can corrupt the rankings.

1. Introduction

Esports tournaments generate a large amount of structured data: who is playing, what map they played on, how many kills they secured and where they finish in the table. Organisers usually track this in spreadsheets, which are easy to break and hard to validate. A small object-oriented program is a good way to model the same domain, because every real-world thing (a player, a match, a tournament) maps neatly to a class.

GameQuest uses **Valorant**, Riot Games' tactical 5v5 shooter, as its theme. Player gamertags (TenZ, zekken, Derke), agents (Jett, Raze, Sage, Sova, Phoenix) and maps (Ascent, Bind, Haven, Split, Icebox, Lotus) are all taken from the game, so the data looks like something a real tournament would produce. The scoring is a simplified model written for this project and is not Riot's official ranking or rating formula.

1.1 Problem statement

Design and implement a system that manages players, tournaments, match scores, rankings and rewards, while supporting different player categories and rejecting invalid scores.

1.2 Objectives

- Model players, matches and tournaments as separate, cooperating classes.
- Use inheritance to create **CasualPlayer** and **ProPlayer** from a common **Player** base class.
- Use polymorphism (virtual functions) so that scoring and reward calculation differ by player type behind the same function names.
- Use operator overloading so that players can be compared, sorted and their scores combined with **+**.
- Use file handling to save tournament results as readable text and as CSV, and to load them back.
- Use exception handling (try, catch and classes derived from std::runtime_error) to reject invalid scores and other bad input.

1.3 Scope

The system is a one-versus-one league: each match is played between two registered players and the combined scores decide the leaderboard. It does not simulate rounds, economy or team play. This keeps the focus on the programming concepts rather than on game mechanics.

1.4 Tools and environment

Item	Details
Language	C++17, standard library only (`iostream`, `fstream`, `memory`, `map`, `algorithm`)
Files produced	`gamequest.cpp` (source), `results.txt`, `results.csv`
Interface	Console / terminal output
Compile and run	`g++ -std=c++17 -Wall -Wextra -o gamequest gamequest.cpp` `./gamequest`

2. System Design

Thedesignhasthreedomain classes (Player with two subclasses, Match, Tournament) and three custom exception classes. The diagram below shows how they relate. Asterisks mark pure virtual functions: the base class declares them and every subclass must implement them.

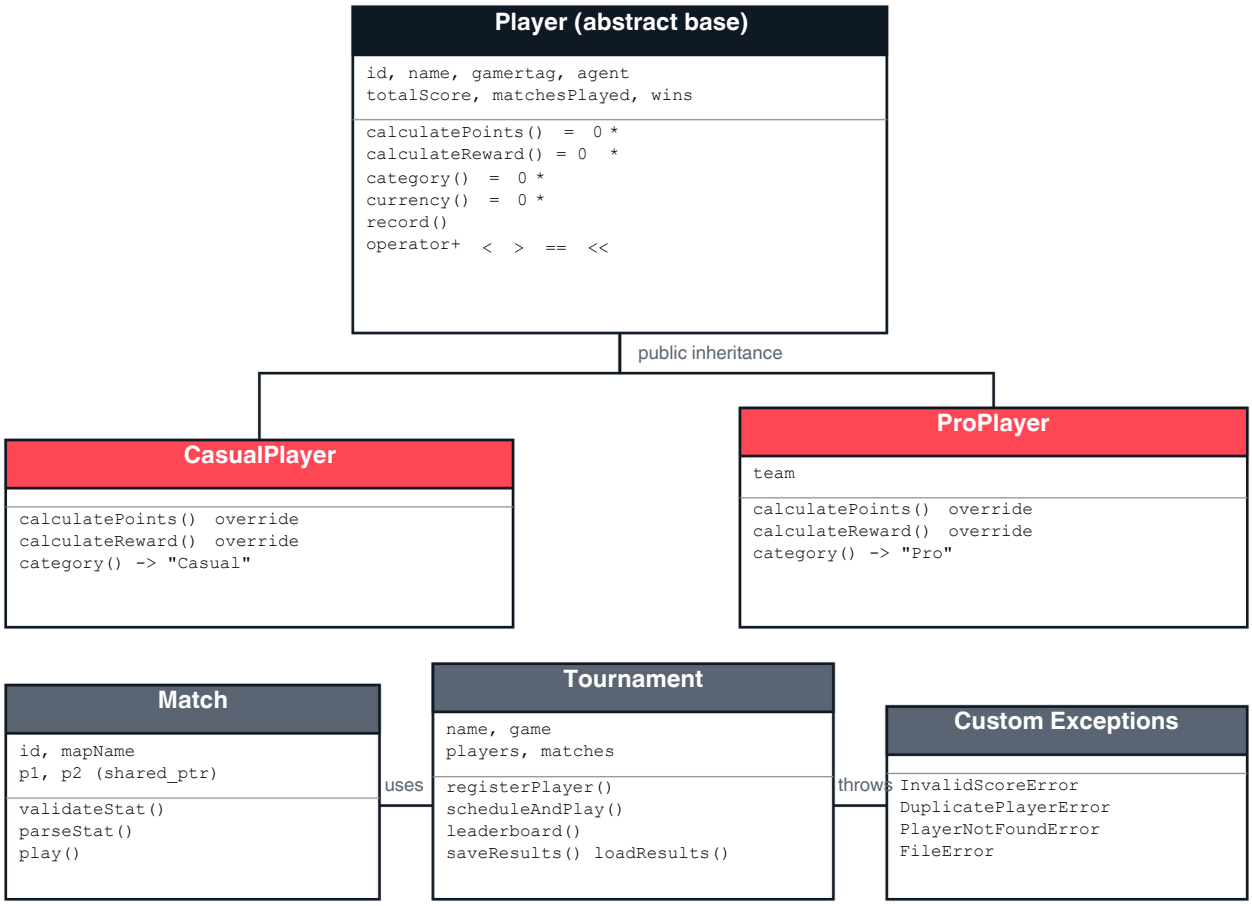


Figure 1: Class diagram of GameQuest

2.1 Responsibilities of each class

Class	Responsibility
Player	Abstract base class. Holds identity (id, name, gamertag, agent) and running totals (score, matches, wins). Declares the scoring and reward interface as pure virtual functions and implements the operators.
CasualPlayer	Simple scoring formula and small credit rewards for everyday players.
ProPlayer	Weighted scoring with a performance bonus, an extra team attribute and USD prize money.
Match	Represents one game on one map. Validates the statistics of both players, decides the winner and records points.
Tournament	Registers players, schedules matches, builds the leaderboard, distributes rewards and saves or loads results.
InvalidScoreError	Thrown for negative, non-numeric or unrealistically large statistics.
DuplicatePlayerError	Thrown when a gamertag is registered twice.

Class	Responsibility
PlayerNotFoundError	Thrown when a match refers to a player id that does not exist.
FileError	Thrown when a results file cannot be opened, written or parsed.

2.2 Program flow

Atournament moves through the same sequence every time. Each step can fail in a controlled way, and the catch blocks keep the program running.

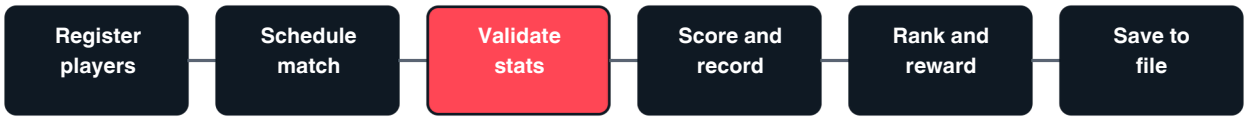


Figure 2: Tournament lifecycle (validation is where invalid scores are rejected)

2.3 Mapping of OOP concepts to the code

Concept	Where it appears	What it achieves
Classes and objects	Player, Match, Tournament	Each real-world entity is one class with its own data and behaviour.
Inheritance	class CasualPlayer : public Player, class ProPlayer : public Player	Shared fields and functions are written once in the base class.
Polymorphism	virtual calculatePoints(), calculateReward(), category(), currency()	The tournament calls the same function through a Player pointer and gets type-specific results at run time.
Operator overloading	operator+, operator<, operator>, operator==, operator<<	Players can be summed, compared, sorted and printed with natural syntax.
File handling	Tournament::saveResults(), loadResults() with ofstream / ifstream	Results survive after the program ends, as .txt and .csv.
Exception handling	InvalidScoreError and three more exception classes; try / catch in main()	Bad input is rejected cleanly instead of crashing or corrupting data.

2.4 Design decisions

- **Validationbeforerecording.** Match::play() validates all six statistics first, so a rejected match never changes any player's score.
- **IDs from a static counter.** Player::nextId gives each player a unique id automatically.
- **Smart pointers.** Players are held in shared_ptr<Player>, so one object can be shared by the roster, the tournament and the matches, and is deleted automatically. The virtual destructor makes deletion through the base pointer safe.
- **Rankings by operator.** leaderboard() calls stable_sort with a comparison that uses the overloaded operator>; equal scores keep their registration order.

3. Scoring and Reward Rules

Each match produces three statistics per player: kills (K), deaths (D) and assists (A). The winner of a match is the player with the higher value of **K + A - D**. If both values are equal, player 1 is declared the winner. The points awarded then depend on the player type.

3.1 Points formulas

Component	CasualPlayer	ProPlayer
Kill value	2 points per kill	3 points per kill
Assist value	1 point per assist	1.5 points per assist
Death penalty	-0.5 per death	-1 per death
Performance bonus	None	+15 if kills >= 25
Win bonus	+10	+25
Minimum	Never below 0	Never below 0

3.2 Reward rules

Final rank	CasualPlayer (credits)	ProPlayer (USD)
1st	500	50,000
2nd	500	25,000
3rd	500	10,000
4th and below	100	1,000

Rewards are based on rank in the combined leaderboard, but the amount and the currency come from the player's own class. That is polymorphism at work: the tournament never asks what type a player is.

3.3 Worked examples (checked against the program output)

Player and match	Calculation	Points
TenZ (Pro), Ascent, won K 24, D 12, A 6	24x3 + 6x1.5 - 12 = 69; no bonus (24 < 25); +25 win	94.0
Derke (Pro), Ascent, lost K 21, D 15, A 8	21x3 + 8x1.5 - 15 = 60; no bonus; no win bonus	60.0
zekken (Pro), Bind, won K 27, D 10, A 4	27x3 + 4x1.5 - 10 = 77; +15 bonus (27 >= 25); +25 win	117.0
AaravX (Casual), Icebox, won K 18, D 11, A 9	18x2 + 9x1 - 11x0.5 = 39.5; +10 win	49.5

4. Implementation

The complete program is listed below in four parts. It is a single source file with no third-party dependencies. The sections follow the design: exceptions first, then the player hierarchy, then match and tournament logic, and finally a demonstration driver that exercises every feature.

4.1 Part A: headers, custom exceptions and the Player base class

```
// GameQuest - Gaming Tournament and Player Ranking System (C++)
// Theme: Valorant Champions-style tournament
// Concepts: Classes, Inheritance, Polymorphism, Operator Overloading,
//           File Handling, Exception Handling
// Build: g++ -std=c++17 -Wall -Wextra -o gamequest gamequest.cpp

#include <algorithm>
#include     <ctime>
#include   <fstream>
#include   <iomanip>
#include  <iostream>
#include <map>
```

```
#include    <memory>
#include   <numeric>
#include   <sstream>
#include <stdexcept>
#include    <string>
#include    <vector>
using namespace std;

// ==== PART A: custom exceptions and the Player base class ===========
class InvalidScoreError : public runtime_error {
public:
    explicit InvalidScoreError(const string& m) : runtime_error(m) {}
};
class DuplicatePlayerError : public runtime_error {
public:
    explicit DuplicatePlayerError(const string& m) : runtime_error(m) {}
};
class PlayerNotFoundError : public runtime_error {
public:
    explicit PlayerNotFoundError(const string& m) : runtime_error(m) {}
};
class FileError : public runtime_error {
public:
    explicit FileError(const string& m) : runtime_error(m) {}
};

class Player {
protected:
    static int nextId;
    int id;
    string name, gamertag, agent;
    double totalScore;
    int matchesPlayed, wins;

public:
    Player(const string& n, const string& g, const string& a)
        : id(nextId++), name(n), gamertag(g), agent(a),
          totalScore(0), matchesPlayed(0), wins(0) {}
    virtual ~Player() {}

    // Polymorphic interface - implemented by each subclass
    virtual double calculatePoints(double k, double d, double a, bool won) const = 0;
    virtual int calculateReward(int rank) const = 0;
    virtual string category() const = 0;
    virtual string currency() const = 0;


    void record(double points, bool won) {
        totalScore += points;
        matchesPlayed++;
        if (won) wins++;
    }
    int getId() const { return id; }
    const string& getGamertag() const { return gamertag; }
    double getScore() const { return totalScore; }
    int getWins() const { return wins; }
    int getMatches() const { return matchesPlayed; }


    // Operator overloading
    double operator+(const Player& o) const { return totalScore + o.totalScore; }
    double operator+(double v) const { return totalScore + v; }
    friend double operator+(double v, const Player& p) { return v + p.totalScore; }
    bool operator<(const Player& o) const { return totalScore < o.totalScore; } bool
    operator>(const Player& o) const { return totalScore > o.totalScore; } bool
    operator==(const Player& o) const { return totalScore == o.totalScore; }


    friend ostream& operator<<(ostream& os, const Player& p) {
        ios::fmtflags saved = os.flags();
        os << "[" << left << setw(6) << p.category() << "] " << setw(12) << p.gamertag
           << " (" << setw(8) << p.agent << ") Score: " << right << fixed
           << setprecision(1) << setw(6) << p.totalScore << "  Wins: " << p.wins
           << "/" << p.matchesPlayed;
        os.flags(saved);
        return os;
    }
};
int Player::nextId = 1;
```

4.2 Part B: CasualPlayer, ProPlayer and Match

```
// ==== PART B: CasualPlayer, ProPlayer and Match =====================
class CasualPlayer : public Player {
public:
    CasualPlayer(const string& n, const string& g, const string& a) : Player(n, g, a) {}
    double calculatePoints(double k, double d, double a, bool won) const override {
        double points = k * 2 + a * 1 - d * 0.5;
        if (won) points += 10;
        return max(points, 0.0);
    }
    int calculateReward(int rank) const override { return rank <= 3 ? 500 : 100; }
    string category() const override { return "Casual"; }
    string currency() const override { return "credits"; }
};


class ProPlayer : public Player {
    string team;
public:
```

```
    ProPlayer(const string& n, const string& g, const string& a, const string& t)
        : Player(n, g, a), team(t) {}
    double calculatePoints(double k, double d, double a, bool won) const override {
        double points = k * 3 + a * 1.5 - d * 1;
        if (k >= 25) points += 15;  // ace-performance bonus
        if (won) points += 25;
        return max(points, 0.0);
    }
    int calculateReward(int rank) const override {
        if (rank == 1) return 50000;
        if (rank == 2) return 25000;
        if (rank == 3) return 10000;
        return 1000;
    }
    string category() const override { return "Pro"; }
    string currency() const override { return "USD"; }
};  const string& getTeam() const { return team; }


struct Stats {
    double kills, deaths, assists;
};
struct MatchResult {
    int id;
    string mapName, p1, p2;
    double pts1, pts2;
    string winner;
};

class Match {
    int id;
    shared_ptr<Player> p1, p2;
    string mapName;

public:
    static constexpr double MAX_STAT = 60;

    Match(inti, shared_ptr<Player> a, shared_ptr<Player> b, const string& m)
        : id(i), p1(a), p2(b), mapName(m) {}

    static stringfmt(double v) {
        ostringstream s;
        s << v;
        return s.str();
    }
    // Converts text to a number, rejecting anything that is not numeric
    static doubleparseStat(const string& text, const string& label) {
        size_t pos = 0;
        double v = 0;
        try {
            v= stod(text, &pos);
        } catch (const exception&) {
            throwInvalidScoreError(label + " must be a number, got '" + text + "'");
        }
        if (pos != text.size())
            throwInvalidScoreError(label + " must be a number, got '" + text + "'");
        return v;
    }
    static void validateStat(double v, const string& label) {
        if (v!= v) throw InvalidScoreError(label + " is not a number");
        if (v< 0) throw InvalidScoreError(label + " cannot be negative (" + fmt(v) + ")");
        if (v> MAX_STAT)
            throwInvalidScoreError(label + " = " + fmt(v) + " exceeds the limit of " + fmt(MAX_STAT));
    }
    static void validate(const Stats& s) {
        validateStat(s.kills, "Kills");
        validateStat(s.deaths, "Deaths");
        validateStat(s.assists, "Assists");
    }

    // The winneris the player with the higher kills + assists - deaths
    MatchResult play(const Stats& s1, const Stats& s2) {
        validate(s1);  // validation happens before anything is recorded
        validate(s2);
        bool win1= (s1.kills + s1.assists - s1.deaths) >= (s2.kills + s2.assists - s2.deaths);
        double pts1 = p1->calculatePoints(s1.kills, s1.deaths, s1.assists, win1);
        double pts2 = p2->calculatePoints(s2.kills, s2.deaths, s2.assists, !win1);
        p1->record(pts1, win1);
        p2->record(pts2, !win1);
        return {id, mapName, p1->getGamertag(), p2->getGamertag(), pts1, pts2,
                win1 ? p1->getGamertag() : p2->getGamertag()};
    }
};
```

Key points in Part B

- The ProPlayerconstructor passes the shared fields to Player through its initializer list and then sets the extra team member.
- The override keyword makes the compiler check that each function really overrides a virtual function of the base class.

- parseStat() converts text with stod and also checks that the whole string was consumed, so text such as "many" or "12abc" is rejected.

4.3 Part C: the Tournament class and file handling

```
// ==== PART C: Tournament and file handling ==========================
class Tournament {
    string name, game;
    map<int, shared_ptr<Player>> players;
    vector<MatchResult> matches;

public:
    Tournament(const string& n, const string& g) : name(n), game(g) {}

    void registerPlayer(shared_ptr<Player> p) {
        for (const auto& kv : players)
            if (kv.second->getGamertag() == p->getGamertag())
                throw DuplicatePlayerError(p->getGamertag() + " is already registered");
        players[p->getId()] = p;
    }
    shared_ptr<Player> getPlayer(int id) const {
        auto it =players.find(id);
        if (it ==players.end()) throw PlayerNotFoundError("No player with id " + to_string(id));
        return it->second;
    }
    MatchResult scheduleAndPlay(int id1, int id2, const string& mapName,
                                const Stats& s1, const Stats& s2) {
        Matchm((int)matches.size() + 1, getPlayer(id1), getPlayer(id2), mapName);
        MatchResult r = m.play(s1, s2);
        matches.push_back(r);
        return r;
    }
    vector<shared_ptr<Player>> leaderboard() const {
        vector<shared_ptr<Player>> v;
        for (const auto& kv : players) v.push_back(kv.second);
        stable_sort(v.begin(), v.end(),
                    [](const shared_ptr<Player>& a, const shared_ptr<Player>& b) { return *a > *b; });
        return v;
    }

    void saveResults(const string& txtPath, const string& csvPath) const {
        auto board = leaderboard();
        time_t now = time(nullptr);
        char date[16];
        strftime(date, sizeof date, "%Y-%m-%d", localtime(&now));

        ofstream txt(txtPath);
        if (!txt)throw FileError("Cannot write '" + txtPath + "'");
        txt << name << " - " << game << "\nDate: " << date << "\n"
            << string(60, '=') << "\nFINAL RANKINGS\n" << fixed << setprecision(1);
        int rank = 1;
        for (const auto& p : board) {
            txt << rank << ". " << left << setw(12) << p->getGamertag() << setw(7) << p->category()
                << right << setw(6) << p->getScore() << " pts  Reward: "
                << p->calculateReward(rank) << " " << p->currency() << "\n";
            rank++;
        }
        txt << "\nMATCH LOG\n";
        for (const auto& r : matches)
            txt <<"M"  << r.id << " [" << r.mapName << "] " << r.p1 << " " << r.pts1 << " vs "
                << r.pts2 << " " << r.p2 << " -> " << r.winner << "\n";

        ofstream csv(csvPath);
        if (!csv)throw FileError("Cannot write '" + csvPath + "'");
        csv << "rank,gamertag,category,score,wins,matches,reward\n" << fixed << setprecision(1);
        rank = 1;
        for (const auto& p : board) {
            csv << rank << "," << p->getGamertag() << "," << p->category() << ","
                << p->getScore() << "," << p->getWins() << "," << p->getMatches() << ","
                << p->calculateReward(rank) << "\n";
            rank++;
        }
    }

    // Returns the number of player rows read; throws FileError on a bad file
    static int loadResults(const string& csvPath) {
        ifstream in(csvPath);
        if (!in) throw FileError("File '" + csvPath + "' not found");
        string line;
        getline(in, line);  // skip header
        int rows = 0;
        while(getline(in, line)) {
            if       (line.empty())       continue;
            stringstream ss(line);
            string field;
            int cols = 0;
            while(getline(ss,  field,  ','))  cols++;
            if (cols != 7)
                throw FileError("File '" + csvPath + "' is corrupted at line " + to_string(rows + 2));
            rows++;
        }
        return rows;
    }
};
```

Key points in Part C

- ofstreamand ifstream close their files automatically when they go out of scope (RAII), even if an exception is thrown.
- loadResults() throws **FileError** with a different message for a missing file and for a damaged row.

4.4 Part D: demonstration driver

```
// ==== PART D: demonstration driver ===================================
int main() {

    cout << fixed << setprecision(1);
    Tournament t("Valorant Champions Cup 2026", "Valorant");

    cout << "--- Registering players ---\n";
    vector<shared_ptr<Player>> roster = {
        make_shared<ProPlayer>("Tyson Ngo", "TenZ", "Jett", "Sentinels"),
        make_shared<ProPlayer>("Aleksander Zygmunt", "zekken", "Raze", "Sentinels"),
        make_shared<ProPlayer>("Nikita Sirmitev", "Derke", "Raze", "Fnatic"),
        make_shared<CasualPlayer>("Aarav Sharma", "AaravX", "Sage"),
        make_shared<CasualPlayer>("Simran Kaur", "SimiOP", "Phoenix"),
        make_shared<CasualPlayer>("Rohan Mehta", "RohanFPS", "Sova"),
    };
    for (const auto& p : roster) {
        t.registerPlayer(p);
        cout << " Registered: " << *p << "\n";
    }
    try {
        t.registerPlayer(make_shared<CasualPlayer>("Fake Dup", "TenZ", "Brimstone"));
    } catch (const DuplicatePlayerError& e) {
        cout << " ! DuplicatePlayerError: " << e.what() << "\n";
    }

    cout << "\n--- Playing matches ---\n";
    struct Fixture { int a, b; string map; Stats s1, s2; };
    vector<Fixture> fixtures = {
         {1, 3, "Ascent", {24, 12, 6}, {21, 15, 8}},
        {2,  4,  "Bind",  {27, 10, 4}, {14, 18, 9}},
        {3,  5,  "Haven", {19, 14, 7}, {16, 16, 5}},
        {1, 2, "Split",  {30, 9, 5},  {22, 13, 10}},
         {4, 6, "Icebox", {18, 11, 9}, {20, 12, 6}},
        {5, 6, "Lotus",   {17, 13, 8}, {12, 17, 4}},
    };
    for (const auto& f : fixtures) {
        MatchResult r = t.scheduleAndPlay(f.a, f.b, f.map, f.s1, f.s2);
        cout << " M" << r.id << " [" << left << setw(6) << r.mapName << "] " << setw(8) << r.p1
             << right << setw(6) << r.pts1 << " vs " << setw(6) << r.pts2 << " " << left
             << setw(8) << r.p2 << " -> Winner: " << r.winner << "\n" << right;
    }

    cout << "\n--- Invalid score tests (Exception Handling) ---\n";
    auto attempt = [&](int a, int b, Stats s1, Stats s2) {
        try {
            t.scheduleAndPlay(a, b, "Ascent", s1, s2);
        } catch (const InvalidScoreError& e) {
            cout << "  ! InvalidScoreError: " << e.what() << "\n";
        } catch (const PlayerNotFoundError& e) {
            cout << "  ! PlayerNotFoundError: " << e.what() << "\n";
        }
    };
    attempt(1, 2, {-5, 10, 3}, {10, 10, 3});
    try { // text input such as "many" is rejected while converting to a number
        Stats bad{Match::parseStat("many", "Kills"), 10, 3};
        attempt(1, 2, {20, 10, 3}, bad);
    } catch (const InvalidScoreError& e) {
        cout << " ! InvalidScoreError: " << e.what() << "\n";
    }
    attempt(1, 2, {99, 10, 3}, {10, 10, 3});
    attempt(1, 99, {10, 10, 3}, {10, 10, 3});

    cout << "\n--- Leaderboard ---\n";
    int rank = 1;
    for (const auto& p : t.leaderboard()) {
        cout << " #" << rank << " " << *p << "   Reward: " << p->calculateReward(rank)
             << " " << p->currency() << "\n";
        rank++;
    }

    cout << "\n--- Operator overloading ---\n";
    const Player &a = *roster[0], &b = *roster[1];
    cout << " " << a.getGamertag() << " + " << b.getGamertag()
    << " (Sentinels team score) = " << a + b << "\n";
    cout << " " << a.getGamertag() << " > " << b.getGamertag() << " ? "
         << (a > b ? "true" : "false") << "\n";
    cout << " " << roster[3]->getGamertag() << " < " << roster[4]->getGamertag() << " ? "
         << (*roster[3] < *roster[4] ? "true" : "false") << "\n";
    double all = accumulate(roster.begin(), roster.end(), 0.0,
                            [](double s, const shared_ptr<Player>& p) { return s + *p; });
    cout << " sum of all players = " << all << "\n";
```

```
    cout<<  "\n---Polymorphism  (same call, different result) ---\n";
    for (int i : {0, 3}) {
        constPlayer& p = *roster[i];
         cout<< "  " << left << setw(6) << p.category() << " " << setw(8) << p.getGamertag()
             << right << " -> points for 20/10/5 win: " << p.calculatePoints(20, 10, 5, true)
             << ",reward  for rank 1: " << p.calculateReward(1) << " " << p.currency() << "\n";
    }

    cout << "\n--- File handling ---\n";
    t.saveResults("results.txt", "results.csv");
    cout << " Saved results.txt and results.csv\n";
    try{
        cout<<  "  Loaded back " << Tournament::loadResults("results.csv") << " player rows\n";
        Tournament::loadResults("missing.csv");
    } catch (const FileError& e) {
        cout<<  "  ! FileError: " << e.what() << "\n";
    }
    return 0;
}
```

5. Concepts in Detail

5.1 Inheritance

CasualPlayerand **ProPlayer** inherit publicly from **Player**: the identity fields, the record() function and the operators are written once. Each subclass adds only what is different. The four functions `calculatePoints()`, `calculateReward()`, `category()` and `currency()` are `declared = 0 in`

the base class, which makes **Player** an abstract class: it cannot be instantiated, and a subclass must implement every one of them.

5.2 Polymorphism

The same call, p.calculatePoints(20, 10, 5, true), returns different values depending on the real type of the object behind the reference. The program output in Section 6 shows this directly: a ProPlayer gets 82.5 points and a CasualPlayer gets 50.0 for identical statistics. Because the functions are virtual, the choice is made at run time. The tournament code contains no if on the player type, so a new category (for example a StreamerPlayer) could be added without changing **Match** or **Tournament**.

5.3 Operator overloading

Operator	Function	Behaviour in GameQuest
a + b	operator+(const Player&)	Returns the combined total score of two players (a team score).
a + 5.0	operator+(double)	Adds a number to a player's score.
sum += *p	friend operator+(double, const Player&)	Lets accumulate() start from 0.0 and add players.
a < b, a > b	operator<, operator>	Compare players by total score. The leaderboard sort `uses operator>`.
a == b	operator==	Two players are equal when their total scores are equal.
cout << a	friend operator<<	Prints a formatted one-line summary used in the leaderboard.

5.4 Exception handling

Exception	Thrown when	Example input
InvalidScoreError	A statistic is negative, not a number, or above the limit of 60	-5, "many", 99
DuplicatePlayerError	A gamertag is already registered in the tournament	second "TenZ"
PlayerNotFoundError	A match uses an id that is not registered	player id 99
FileError	A results file is missing, cannot be written, or has a damaged row	missing.csv

All four classes derive from std::runtime_error, so each carries a message available through what(), and the caller can catch them individually or all together. In the demo, each error is caught, printed and the program continues with the next match.

6. Program Output

The following is the real, unedited output of running `./gamequest`.

```
--- Registering players ---

 Registered: [Pro    ] TenZ         (Jett    ) Score:    0.0  Wins: 0/0
 Registered: [Pro    ] zekken       (Raze    ) Score:    0.0  Wins: 0/0
 Registered: [Pro    ] Derke        (Raze    ) Score:    0.0  Wins: 0/0
   Registered:   [Casual]   AaravX    (Sage    ) Score:    0.0  Wins: 0/0
Registered:    [Casual]    SimiOP    (Phoenix ) Score:    0.0  Wins: 0/0
Registered: [Casual] RohanFPS       (Sova    ) Score:    0.0  Wins: 0/0
 ! DuplicatePlayerError: TenZ is already registered

--- Playing matches ---
M1 [Ascent] TenZ        94.0 vs   60.0   Derke  -> Winner: TenZ
 M2 [Bind ] zekken   M3117.0 vs   28.0  AaravX  -> Winner: zekken
[Haven ] Derke          78.5 vs   29.0  SimiOP  -> Winner: Derke
 M4 [Split ] TenZ      128.5 vs   68.0 zekken   -> Winner: TenZ
 M5 [Icebox] AaravX  M6 49.5 vs   40.0 RohanFPS -> Winner: AaravX
[Lotus ] SimiOP         45.5 vs   19.5 RohanFPS -> Winner: SimiOP

--- Invalid score tests (Exception Handling) ---
 ! InvalidScoreError: Kills cannot be negative (-5)
 ! InvalidScoreError: Kills must be a number, got 'many'
! InvalidScoreError: Kills = 99 exceeds the limit of 60  !
PlayerNotFoundError: No player with id 99

--- Leaderboard ---
 #1 [Pro    ] TenZ         (Jett   )   Score:  222.5  Wins: 2/2  Reward:  50000  USD
 #2 [Pro    ] zekken       (Raze   )   Score:  185.0  Wins: 1/2  Reward:  25000  USD
 #3 [Pro    ] Derke        (Raze   )   Score:  138.5  Wins: 1/2  Reward:  10000  USD
   #4  [Casual]  AaravX     (Sage   )   Score:   77.5  Wins: 1/2  Reward: 100 credits
#5 [Casual] SimiOP  #6     (Phoenix ) Score:   74.5  Wins: 1/2  Reward: 100 credits
[Casual] RohanFPS          (Sova ) Score:      59.5  Wins: 0/2  Reward: 100 credits

--- Operator overloading ---
 TenZ + zekken (Sentinels team score) = 407.5
TenZ > zekken ? true
 AaravX < SimiOP ? false
 sum of all players = 757.5

--- Polymorphism (same call, different result) ---
 Pro     TenZ     -> points for 20/10/5 win: 82.5, reward for rank 1: 50000 USD
 Casual AaravX    -> points for 20/10/5 win: 50.0, reward for rank 1: 500 credits

--- File handling ---
 Saved results.txt and results.csv
 Loaded back 6 player rows
 ! FileError: File 'missing.csv' not found
```

6.1 Leaderboard chart

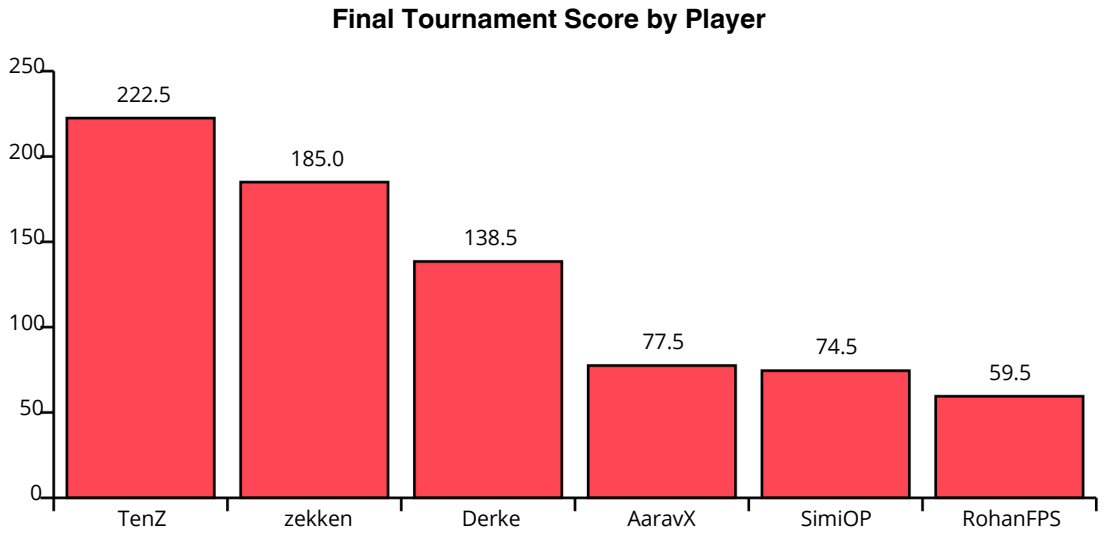


Figure 3: Final scores. The three ProPlayers lead because of their heavier weighting and bonuses.

6.2 Reading the results

- **TenZ**finishesfirstwith222.5 points after winning both of his matches, including a 128.5-point win against zekken.
- The three Pro players take the top three places, so they receive the 50,000, 25,000 and 10,000 USD prizes.
- All three Casual players receive the participation reward of 100 credits, because none finished in the top three.
- Of the Pro-versus-Casual comparison, the same 20/10/5 performance is worth 82.5 for a Pro and 50.0 for a Casual player: the weighting is the main reason the ranking separates by category.
- The **+** operator gives the Sentinels pair (TenZ and zekken) a combined 407.5, and sum() over all six players gives 757.5.

7. Saved Files

7.1 results.txt (human-readable report)

```
ValorantChampions  Cup 2026 - Valorant
Date:                                                2026-10-28
============================================================
FINAL RANKINGS
1. TenZ         Pro Pro 222.5 pts  Reward:  50000  USD
2. zekken       Pro     185.0 pts  Reward:  25000  USD
3. Derke        Casual  138.5 pts  Reward:  10000  USD
4. AaravX       Casual   77.5 pts  Reward: 100 credits
5. SimiOP       Casual   74.5 pts  Reward: 100 credits
6. RohanFPS              59.5 pts  Reward: 100 credits

MATCH LOG
M1 [Ascent] TenZ 94.0 vs 60.0 Derke -> TenZ
M2 [Bind] zekken 117.0 vs 28.0 AaravX -> zekken
M3 [Haven] Derke 78.5 vs 29.0 SimiOP -> Derke
M4 [Split] TenZ 128.5 vs 68.0 zekken -> TenZ
M5 [Icebox] AaravX 49.5 vs 40.0 RohanFPS -> AaravX
M6 [Lotus] SimiOP 45.5 vs 19.5 RohanFPS -> SimiOP
```

7.2 results.csv (comma-separated values)

```
rank,gamertag,category,score,wins,matches,reward
1,TenZ,Pro,222.5,2,2,50000
2,zekken,Pro,185.0,1,2,25000
3,Derke,Pro,138.5,1,2,10000
4,AaravX,Casual,77.5,1,2,100
5,SimiOP,Casual,74.5,1,2,100
6,RohanFPS,Casual,59.5,0,2,100
```

The text file is meant to be read or printed. The CSV file keeps the player table in a form that opens directly in Excel or Google Sheets or can be read by another program; loadResults() in the demo reads it back, checks that every row has seven fields, and reports 6 player rows.

8. Testing

Each feature was tested by running the demonstration with both valid and invalid input. Every result below is taken from the program output in Section 6.

#	Test	Input	Expected	Result
1	Register six valid players	3 Pro + 3 Casual	All registered	Pass
2	Duplicate gamertag	Second 'TenZ'	DuplicatePlayerError	Pass
3	Negative kills	-5	InvalidScoreError	Pass
4	Non-numeric kills	'many'	InvalidScoreError	Pass
5	Unrealistic kills	99 (limit 60)	InvalidScoreError	Pass
6	Unknown player id	99	PlayerNotFoundError	Pass
7	Rejected match leaves scores unchanged	Tests 3 to 6	Leaderboard shows only 6 valid matches	Pass
8	Leaderboard order	After 6 matches	Descending by score	Pass
9	Polymorphic points	20/10/5, win	Pro 82.5, Casual 50.0	Pass
10	Polymorphic rewards	Rank 1	Pro 50000 USD, Casual 500 credits	Pass
11	Operator +	TenZ + zekken	222.5 + 185.0 = 407.5	Pass
12	accumulate() over players	All six	757.5	Pass
13	Save and reload	results.csv	6 player rows read back	Pass
14	Missing file	missing.csv	FileError caught, no crash	Pass

8.1 Known limitations

- If twoplayers haveequaltotal scores they compare as equal and keep their original order; there is no tie-break rule yet.
- An exact tie inside a match is awarded to player 1.
- Matches are one-versus-one; there is no round-by-round or team simulation.
- Results are written only when saveResults() is called; there is no automatic saving.
- The scoring formula is a simplified model made for this project, not an official Valorant rating.

9. Conclusion and Future Scope

GameQuest showshow the main ideasof object-oriented programming fit together in one small but realistic system. Inheritance removed duplicated code between player types, polymorphism let the tournament treat every player the same way while still producing different scores and rewards, operator overloading made ranking and team totals read naturally, file handling saved the results for later use, and exception handling stopped bad scores from entering the rankings. The Valorant theme made the data realistic and the output easy to follow.

9.1 Future enhancements

- Add a tie-break rule such as wins first, then kill-death ratio.
- Add a TeamPlayer or StreamerPlayer subclass to demonstrate extensibility.
- Support knockout brackets and round-robin scheduling instead of manually listed fixtures.
- Replace the console with a simple GUI (for example Qt) or a web page that reads `results.csv`.
- Store data in a database such as SQLite for multiple tournaments.
- Add unit tests with Google Test or Catch2 to automate Section 8.

References

- cppreference.com. Virtual functions, Abstract classes and Operator overloading. en.cppreference.com
- cppreference.com. std::runtime_error, try-catch blocks and std::basic_fstream. en.cppreference.com
- cppreference.com. std::shared_ptr, std::stable_sort and std::accumulate. en.cppreference.com
- Riot Games. Valorant (agents and maps used as theme data). playvalorant.com